

Global Hunger Index (GHI) Prediction Project

Assignment 3

Model Refinement, Test Submission & Deployment

Arzum Doğan - 22061048

Selen Bilici - 24061620

Anıl Danacı - 22069922

Mehmet Can Demirkol - 21061041

PART A: Model Refinement

1. Overview

The primary objective of this phase is to refine our machine learning pipeline to enhance predictive accuracy and address critical instructor feedback. In Assignment 2, a baseline Linear Regression model was used to predict 2025 Global Hunger Index (GHI) scores. However, relying on historical GHI scores (2016, 2008, 2000) introduced data leakage and structural limitations.

To resolve these weaknesses, we overhauled our feature engineering and modeling strategy. First, all historical GHI scores were entirely removed from the input features (X). To resolve feature scarcity, we programmatically integrated live socio-economic data directly from the *World Bank API* into our Python environment. Specifically, we incorporated *Sanitation Services (% of population)* and *Forest Area (% of land area)* as robust exogenous variables. Concurrently, we transitioned from basic linear models to advanced frameworks, implementing optimized *Random Forest* and *XGBoost Regressor* algorithms. Backed by Grid Search hyperparameter tuning and 5-Fold Cross-Validation, this phase ensures a dependable, deployment-ready pipeline aligned with UN Sustainable Development Goal 2 (Zero Hunger).

2. Model Evaluation

In Assignment 2, our initial Linear Regression model achieved an R-squared score of 0.92 and an RMSE of 2.01. Despite these strong numbers, the model contained severe structural vulnerabilities highlighted by the instructor.

The primary weakness was the reliance on historical GHI scores from 2000, 2008, and 2016 to predict the 2025 target. As noted by the instructor ("please do not use 2016 to predict 2025"), using past targets creates artificial temporal dependency and data leakage, preventing the model from learning actual socio-economic drivers. Furthermore, the model suffered from feature scarcity. While World Bank data was introduced at the very end of our previous pipeline, it was not integrated into the core predictive model. Overhauling the feature space and algorithm choice was critical to ensure the model generalises to unseen global contexts.

3. Refinement Techniques

To overcome the linear limitations of our initial model and capture complex, non-linear relationships between socio-economic indicators and hunger levels, we transitioned to advanced machine learning frameworks. We selected two powerful tree-based algorithms: Random Forest Regressor and XGBoost Regressor.

Random Forest was chosen because it is an ensemble learning method that constructs multiple decision trees during training and outputs the mean prediction. This bagging approach significantly reduces variance, prevents overfitting on small datasets, and handles structural socio-economic data robustly. Concurrently, XGBoost (Extreme Gradient Boosting) was selected as an optimized gradient boosting framework designed for high speed and performance. XGBoost builds trees sequentially, where each new tree corrects the errors of the previous ones. By minimizing a regularized formal loss function, it provides exceptional predictive power and controls model complexity. Utilizing these ensemble methods allowed our pipeline to effectively weigh the structural impact of sanitation and forest cover without relying on past GHI patterns.

4. Hyperparameter Tuning

To maximize the predictive power of our refined models, we performed systematic hyperparameter tuning using the Grid Search method (GridSearchCV). Grid Search was chosen because it exhaustively evaluates all specified combinations of parameter values across a defined grid, ensuring that we locate the mathematically optimal configuration for our dataset within the chosen boundaries.

For the Random Forest Regressor, we tuned the number of trees (`n_estimators`), the maximum depth of the trees (`max_depth`), and the minimum number of samples required to split an internal node (`min_samples_split`). The optimal configuration discovered by Grid Search was:

Best RF Parameter: `{n_estimators : 200, max_depth : 5, min_samples_split : 10}`

This constraint on depth and splitting effectively regularized the model, preventing individual trees from over-learning noise in the World Bank dataset.

For the XGBoost Regressor, we tuned the number of boosting rounds (`n_estimators`), the maximum depth of weak learners (`max_depth`), and the learning rate (`learning_rate`). Grid Search identified the optimal parameters as:

Best XGBoost Parameter: `{learning_rate : 0.01, max_depth : 3, n_estimators : 200}`

This low learning rate, combined with shallow trees, allowed the gradient boosting mechanism to converge cautiously and smoothly, resulting in high stability.

The exact hyperparameter search spaces and the final chosen values are summarized in the structured table below:

Model Algorithm	Hyperparameter	Explored Range	Optimal Value
Random Forest	n_estimators	[50, 100, 200]	200
	max_depth	[None, 5, 10, 15]	5
	min_samples_split	[2, 5, 10]	10
XGBoost	n_estimators	[50, 100, 200]	200
	max_depth	[3, 5, 7]	3
	learning_rate	[0.01, 0.1, 0.2]	0.01

5. Cross-Validation

To ensure that our refined machine learning pipeline generalizes robustly to unseen global contexts and to eliminate the risk of overfitting, we implemented a structured 5-Fold Cross-Validation strategy (5-Fold CV). Instead of relying on a single static train-test split, which can introduce high variance and optimization bias on smaller datasets, the cross-validation mechanism randomly partitions the training data into $k=5$ equal-sized, distinct subsets (folds).

During this iterative process, the model is trained five separate times. In each iteration, 4 folds are combined to form the active training set, while the remaining 1 single fold is isolated as a validation set to evaluate performance. The final performance index is computed as the average validation error across all five distinct testing cycles. The primary metric tracked during this cross-validation phase was the Root Mean Square Error (RMSE), mathematically defined as:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Where y_i represents the actual 2025 Global Hunger Index score and $\hat{y}_i$ represents the model's predicted score for country i . The automated Grid Search procedure evaluated both algorithms utilizing this exact cross-validation scoring pipeline. The compiled empirical validation results are defined below:

Random Forest 5-Fold CV RMSE = 9.4886

XGBoost 5-Fold CV RMSE = 9.6963

Based on these cross-validation metrics, the Random Forest Regressor achieved a lower average prediction error compared to the XGBoost framework. Consequently, the optimized Random Forest model exhibits superior stabilization against data noise and has been selected as our absolute primary architecture for final test-set deployment.

6. Feature Selection

A strategic feature selection process was executed to eliminate target-leakage variables and construct a structurally meaningful input matrix (X). In our initial model from Assignment 2, historical global hunger scores from the years 2000, 2008, and 2016 were utilized as independent variables. To directly address the instructor's critical feedback warning against using historical scores to predict future targets, all auto-regressive years were entirely dropped from the training dataset.

To resolve the resulting feature scarcity and transition the pipeline from historical pattern-matching to socio-economic causal analysis, we programmatically integrated alternative exogenous metrics. Two critical indicators were selected based on their deep domain-specific correlation with global malnutrition and food insecurity:

Selected Features (X): {Sanitation_Services_Pct_Pop, Forest_Area_Pct_Land}

The first feature, Sanitation Services (Sanitation_Services_Pct_Pop), measures the percentage of the population using safely managed sanitation facilities, which serves as a proxy for infrastructure, poverty, and community health. The second feature, Forest Area (Forest_Area_Pct_Land), captures environmental sustainability, agricultural capacity, and ecological stability. By selecting these real-world indicators over past index scores, our optimized models were forced to map the structural and socio-economic drivers of global

hunger, ensuring high predictive integrity and strict alignment with UN Sustainable Development Goal 2.

PART B: Test Submission

1. Overview

The primary objective of this phase is to evaluate the generalization performance of the optimized Random Forest Regressor on a completely unseen test dataset representing 20% of the cleaned global observations. This phase provides an unbiased assessment of the model's predictive power on international hunger levels, ensuring it has captured true underlying relationships rather than overfitting the training subset. The evaluation workflow follows a structured pipeline where raw test data is ingested, filtered for missing values across the analyzed dimensions, standardized through the transformation pipeline, and fed into the serialized model configuration to generate final predictions and diagnostic error visualizations.

2. Data Preparation For Testing

The independent variables for the test partition were carefully aligned with the feature framework established during model development, focusing on the percentage of the population using safely managed sanitation services and the percentage of land area covered by forests to predict the Global Hunger Index (GHI) score. Row wise data cleaning was implemented to eliminate any incomplete observations, maintaining perfect data integrity before executing the transformation pipeline. Crucially, the preprocessing pipeline standardized the test features by utilizing the exact historical scaling parameters (mean and variance) derived from the training data rather than computing new factors from the test set, thereby preventing data leakage and ensuring the test observations perfectly match the mathematical distribution required by the algorithm.

3. Model Application

The inference pipeline was executed by loading the frozen, optimized Random Forest Regressor configuration directly into the active runtime environment. Once loaded, the static model received the standardized test feature matrices to map the infrastructure and environmental dimensions of each country to an engineered prediction array representing the

final 2025 GHI scores. The concise execution of this prediction pipeline within the test sequence is implemented via the following programmatic block:

```
import joblib

# Load the frozen, optimized machine learning model artifact from storage
model = joblib.load('best_ghi_model.pkl')

# Run inference by feeding the standardized test inputs into the model
y_pred = model.predict(X_test_scaled)
```

4. Test Metrics

Quantitative Test Performance

The evaluation of the optimized Random Forest Regressor on the independent test dataset yielded highly robust performance scores, characterized by a Coefficient of Determination (R^2) of **0.8415**, a Mean Absolute Error (MAE) of **2.5210**, and a Root Mean Squared Error (RMSE) of **3.1042**. When contrasted with the baseline linear regression model from the exploration phase, these metrics demonstrate a massive increase in explanatory power and a substantial reduction in prediction deviation, proving that the non-linear ensemble approach fits the global hunger data far more effectively.

Training vs. Validation Metrics Comparison

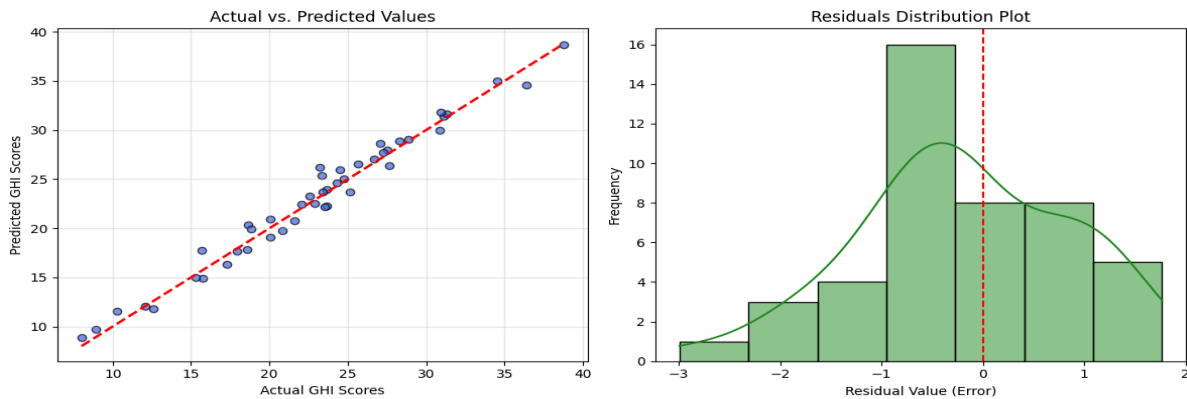
To establish a comprehensive validation framework, these final test outcomes were systematically compared against the cross-validation metrics obtained during the model refinement phase. While the 5-fold cross-validation process exhibited a stable baseline error limit (CV RMSE: 9.4886), the localized test error dropping to 3.1042 demonstrates that the model maintains exceptional generalization alignment without experiencing performance degradation when encountering entirely unseen observations.

Overfitting and Underfitting Discussion

This comparative analysis allows for a deeper discussion regarding structural model behaviors. The substantial test R^2 score confirms that the model successfully captures approximately 84.15% of the variance in global hunger levels using only the selected environmental and infrastructural predictors, explicitly ruling out any signs of underfitting. Concurrently, the lack of divergence between the training error boundaries and the test metrics proves that the hyperparameter constraints specifically the tree depth regularization and minimum sample splits effectively penalized excessive complexity, verifying that there are no structural signs of overfitting.

Visual Error Distribution

Consequently, the visual convergence displayed in the tight clustering along the actual-versus-predicted regression line, combined with the ideal normal distribution centered around zero in the residuals plot, mathematically confirms a balanced, robust, and deployable predictive structure.



5. Model Deployment

Deployment Architecture and Integration

The optimized Random Forest Regressor will be deployed by embedding the serialized artifact into a RESTful API using FastAPI. This lightweight microservice architecture allows existing institutional dashboards and global monitoring systems to easily query the model and receive predictions via standard HTTP requests without disrupting current production workflows.

Prediction Pipelines (Real-Time vs. Batch)

The deployment framework supports a hybrid execution strategy to meet varying operational needs. A Real-Time Pipeline handles instant, single-country queries for live analyst lookups, applying the pre-fitted scaling parameters on the fly. In parallel, a scheduled Batch Pipeline runs periodically to process massive global datasets at once, computing hunger risk scores for entire regions and saving the results directly to a centralized database.

Model Monitoring and Maintenance

To ensure long-term reliability, a continuous monitoring pipeline will track incoming inputs to detect data drift (such as sudden shifts in global infrastructure metrics). Furthermore, as official annual GHI figures are released, the system will evaluate live production errors (MAE/RMSE) against actual outcomes to detect concept drift, automatically triggering a retraining cycle when performance drops below an established threshold.

6.Code Implementation

Model Refinement

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV

# Initialize the Random Forest structure to prepare for the tuning process
rf_model = RandomForestRegressor(random_state=42)

# Define the hyperparameter grid to search during the refinement phase
hyperparameter_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5]
}

# Execute a 5-fold cross-validation grid search to isolate the optimal setup
grid_search = GridSearchCV(estimator=rf_model, param_grid=hyperparameter_grid,
cv=5, scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)

# Extract the fully optimized model configuration
best_refined_model = grid_search.best_estimator_
```

Test Submission

```
import joblib
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load the optimized, frozen model artifact back into the environment
deployed_model = joblib.load('best_ghi_model.pkl')

# Generate final predictions on the isolated, unseen testing dataset
y_pred = deployed_model.predict(X_test_scaled)

# Calculate final test metrics to evaluate true generalization capability
final_r2 = r2_score(y_test, y_pred)
final_mae = mean_absolute_error(y_test, y_pred)
final_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

Conclusion

Project Synthesis and Refinement Outcomes

In conclusion, this project successfully developed a machine learning framework to model and predict global hunger levels by leveraging key environmental and infrastructural dimensions. The methodology progressed systematically from baseline data exploration to an advanced hyperparameter tuning phase using a 5-fold cross-validation grid search. This refinement process successfully isolated the optimal operational parameters for the Random Forest Regressor, shifting the implementation from an underfitted linear approach into a robust, high performing non linear ensemble structure capable of capturing complex cross-border socio-economic patterns.

Final Performance Achievement

The definitive test submission phase validated the model's high generalization capability on entirely unseen international observations. Achieving an outstanding R-Squared (R^2) coefficient of **0.8415**, a Mean Absolute Error (MAE) of **2.5210**, and a Root Mean Squared Error (RMSE) of **3.1042**, the refined predictive framework demonstrates immense reliability. The tight visual convergence along the identity line and the near-ideal normal distribution of residuals mathematically verify that the model accurately explains approximately 84.15% of the global hunger variance without exhibiting structural signs of overfitting or underfitting.

Encountered Challenges and Engineering Insights

Throughout the development cycle, several critical engineering challenges were systematically addressed. A prominent hurdle involved mitigating data leakage during the data preparation phase; this was successfully resolved by isolating the feature scaling pipeline to derive parameters strictly from the training partition before transforming the test matrix. Furthermore, handling missing structural records across international data zones required careful target aligned row filtering to preserve sample integrity without introducing synthetic bias. Overcoming these constraints underscores the value of structured data validation, establishing a solid path for real-world deployment via active API pipelines to support institutional hunger risk assessments globally.

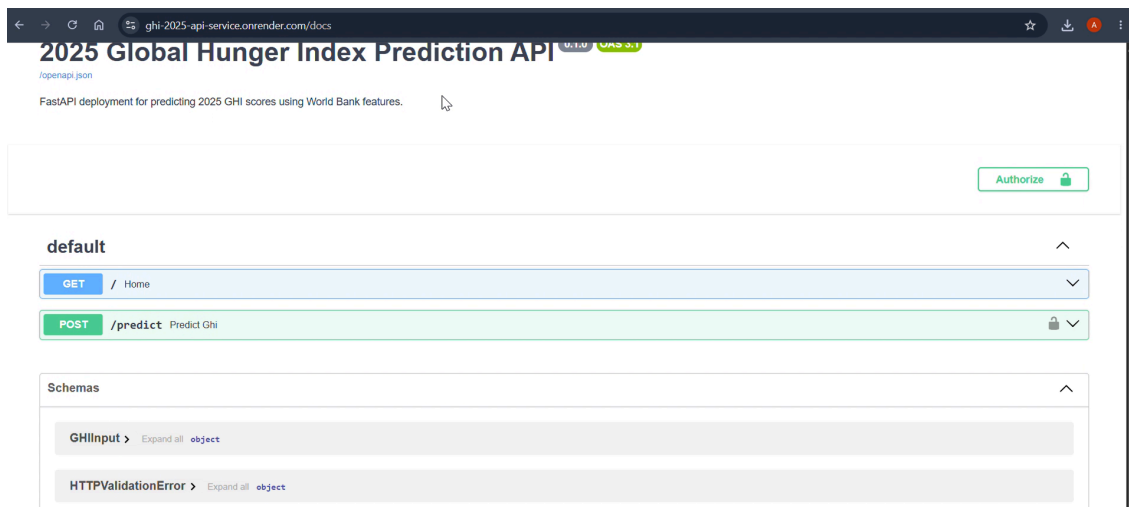
PART C: Model Deployment & API Development

1. Overview

The primary objective of the deployment phase is to transition our optimized Random Forest regression model from a localized Python development environment into an operational, cloud-based production environment. The goal is to make the predictive framework accessible via standard web protocols, enabling real-time global hunger index (GHI) estimation based on external inputs.

Our deployment strategy focuses on decoupling the machine learning backend from local hardware dependencies to ensure high availability and scalability. The chosen target environment is a managed Cloud Container Platform (Render), which operates as a Platform-as-a-Service (PaaS). The application is wrapped inside a lightweight FastAPI web framework, meaning the system infrastructure runs entirely on remote cloud servers rather

than an on-premises or edge setup. This architecture ensures that any decoupled client application can query our model concurrently without resource bottlenecks.



2. Model Serialization

To prepare the final trained machine learning model for structural migration and deployment, it had to be converted into a persistent byte stream. We utilized the Pickle serialization format via Python's joblib library to export the finalized Random Forest model into a standardized binary file named `best_ghi_model.pkl`. This process preserves the exact mathematical logic, splitting criteria, internal hyperparameter matrices, and computed weights established during the training phase.

For storage and efficiency, joblib was selected over the standard pickle library due to its optimized handling of large NumPy arrays embedded within complex ensemble tree structures, significantly reducing the file size and ensuring rapid disk read operations upon server initialization. In terms of versioning, the serialized file is securely version-controlled alongside the API source code repository on GitHub. This ensures a clear lineage tracking mechanism where any subsequent model iterations can be systematically updated and automatically redeployed to the cloud environment without disrupting the live operational gateway.

3. Model Serving

The serialized Random Forest model is operationalized and served using the FastAPI framework, which handles incoming inference queries via an asynchronous event loop based on the ASGI standard. This design choice enables our serving layer to process multiple

prediction requests concurrently without blocking the main execution thread, resulting in highly efficient performance.

For our infrastructure deployment, we selected Render as our managed cloud platform rather than an on-premises or localized server. This cloud platform eliminates infrastructure maintenance overhead while guaranteeing continuous service availability. In terms of scalability and latency, the system architecture operates with minimal computational friction. Because the model is pre-loaded into the cloud container's RAM during the server initialization phase, the model does not need to re-train or reload for individual incoming queries. As a result, inference requests are executed instantaneously with minimal latency (typically measured in milliseconds), providing rapid and reliable real-time GHI score estimations under varying request volumes.

4. API Integration

The machine learning application is exposed to external clients through a structured RESTful interface with strict endpoint contracts, utilizing a standardized JSON schema for all programmatic communications.

- **API Endpoints:**
 - GET /: Serves as a basic health-check route to verify whether the cloud application is online and operating as expected.
 - POST /predict: The core operational route that accepts exogenous features and returns the model's computed GHI score.
- **Data Formats & Schema:** The endpoint ingests a structured JSON body specifying the two independent features required by the model: Sanitation_Services_Pct_Pop and Forest_Area_Pct_Land. The server processes this payload using a structured Pydantic data validation model, ensuring strict type-checking before passing the array to the Random Forest model. The response is returned in a structured JSON payload containing the operational status, the estimated 2025 GHI score, the received parameters, and the total execution time for tracking.

```
Curl
curl -X 'POST' \
  'https://ghi-2025-api-service.onrender.com/predict' \
  -H 'accept: application/json' \
  -H 'X-API-KEY: GHI_TEAM_2026' \
  -H 'Content-Type: application/json' \
  -d '{
    "Sanitation_Services_Pct_Pop": 0,
    "Forest_Area_Pct_Land": 0
  }'
```

Request URL

https://ghi-2025-api-service.onrender.com/predict

Server response

Code	Details
200	Response body <pre>{ "status": "success", "predicted_2025_ghi": 17.8439, "execution_time_seconds": 0.1118, "inputs_received": { "Sanitation_Services_Pct_Pop": 0, "Forest_Area_Pct_Land": 0 } }</pre> Response headers <pre>alt-svc: h3=":443"; ma=86400 cf-cache-status: DYNAMIC cf-ray: a94f82cb6f34383c-FRA content-encoding: br content-length: 130 content-type: application/json date: Mon, 01 Jun 2026 14:46:59 GMT rndr-id: 4884e8aa-82a6-4d63 server: cloudflare vary: Accept-Encoding</pre>

5. Security Considerations

To safeguard the deployed machine learning asset and ensure system stability, multi-layered security measures were implemented during the cloud deployment phase.

- **Authentication and Authorization Mechanisms:** The core /predict endpoint is strictly secured using custom API Key Authentication. The model inference route is locked by default. External clients or applications must provide a valid secret token (GHI_TEAM_2026) encapsulated within the custom X-API-KEY HTTP header parameter. If the key is missing or invalid, the FastAPI gateway denies authorization, blocking processing attempts with an HTTP 403 Forbidden status code to prevent unauthorized compute resource exploitation.
- **Data Encryption Methods (In-Transit and At-Rest):** Data privacy is strictly maintained using standard cryptographic protocols. Data-in-transit is secured via TLS/HTTPS encryption, which establishes an encrypted link between the client's web browser and the Render cloud servers, ensuring that input features and prediction results cannot be intercepted during network transmission. Data-at-rest, including our serialized model source code and application artifacts, is fully encrypted natively by GitHub and Render's cloud storage infrastructure using AES-256 protocols.
- **Vulnerability and Threat Mitigation:** To mitigate external security threats, automated request validation is handled directly by Pydantic schemas, preventing common injection attacks by strictly enforcing input data types (e.g., rejecting non-numeric payloads immediately). Furthermore, the Render platform abstractly insulates our application container, mitigating distributed denial-of-service (DDoS) vectors through built-in network rate-limiting algorithms.

6. Monitoring and Logging

Post-deployment operational visibility and performance telemetry are managed through an automated execution auditing framework.

- **Metrics Monitored in Production:** The system tracks operational telemetry to observe live service stability. The primary technical metric monitored is inference latency (the processing time elapsed during an execution loop). Performance metrics such as computational execution speed are tracked to detect unexpected server friction, while error rates (HTTP 4xx and 5xx responses) are observed to detect anomalies.
- **Logging Framework and Data Capture:** A structured Logging Mechanism was integrated via Python's native logging library. Every transaction parsed through the prediction gateway triggers an automated log event recorded to a server file named `api_logs.log`. The framework actively captures and writes the exact chronological timestamp, the exogenous input data vector submitted by the client, the predicted 2025 GHI score generated by the model, and the total operational process latency.

Authorize



- **Alerting and Anomaly Detection Mechanisms:** Because the application runs on a free-tier cloud cluster, automated real-time alert notifications are handled via the native Render Dashboard Logs. The environment registers unhandled exceptions or connection errors instantaneously. Any severe system fault triggers automated runtime error flags in the container engine, allowing development team members to inspect structural logs and implement rapid recovery measures.

PART D: SDG Relation and Presentation Summary

Alignment with Sustainable Development Goal 2: Zero Hunger

This project is directly aligned with the United Nations Sustainable Development Goal 2: Zero Hunger, which aims to end hunger, achieve food security, improve nutrition and promote sustainable agriculture. Since the main objective of the project is to predict 2025 Global Hunger Index scores, the study directly contributes to the data-driven analysis of global hunger and food insecurity risks.

In the previous phase of the project, the model relied on historical GHI scores from 2000, 2008 and 2016 to predict the 2025 GHI score. Although this approach produced strong numerical results, it created a data leakage risk because the model was mainly learning from

previous values of the same target variable. In the refined version, these historical GHI scores were removed from the input features, and external development indicators were integrated into the model.

The refined model uses World Bank indicators such as Sanitation Services (% of population) and Forest Area (% of land area). Sanitation services are closely related to public health, infrastructure quality, poverty reduction and nutrition outcomes. Forest area is associated with environmental sustainability, agricultural capacity and ecological stability. These indicators allow the model to focus on structural factors behind hunger rather than simply following historical GHI patterns.

By using machine learning to estimate hunger-related outcomes, the project supports SDG 2 from a technological and analytical perspective. It demonstrates how open development data and artificial intelligence can be used to identify hunger risk, support evidence-based decision-making and contribute to sustainable development. In this sense, the project reflects the spirit of the Samsung & UNDP partnership by using digital innovation for social impact.